

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет по лабораторной работе №1
Тема: «Среда разработки Code Composer Studio. Плата MSP-
EXP430F5529. Цифровой ввод-вывод»

Выполнил:
студент группы 250541 Власов Р. Е.

Проверил:
к.т.н., доцент _____ Селезнёв И. Л.

Минск
2025

1. ЦЕЛЬ РАБОТЫ

Изучить принципы работы с цифровым вводом-выводом платы MSP-EXP430F5529 и ознакомиться с интегрированной средой разработки Code Composer Studio.

2. ИСХОДНЫЕ ДАННЫЕ К РАБОТЕ

1. Интегрированная среда разработки Code Composer Studio.
2. Плата MSP-EXP430F5529.
3. Программа должна работать следующим образом:
 - после запуска должны гореть 2 светодиода;
 - после первого нажатия на кнопку – первый светодиод не горит, а второй мигает;
 - после второго нажатия на кнопку – первый мигает, а второй не горит;
 - после третьего нажатия на кнопку – горят оба светодиода;
 - режимы работы должны меняться по отпусканию кнопки;
 - программная обработка дребезга кнопки.

3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

3.1 Интегрированная среда разработки Code Composer Studio

Разработка проектов для лабораторного макета MSP-EXP430F5529 проводится в интегрированной среде разработки Code Composer Studio.

Code Composer Studio (или сокращенно CCS) — интегрированная среда разработки для создания кода для DSP и/или ARM процессоров семейства TMS320, и других процессоров, таких как MSP430, выпускаемых Texas Instruments. Code Composer Studio включает операционную систему реального времени DSP/BIOS, позднее получившую название SYS/BIOS.

CCS представляет собой единый графический интерфейс для пошаговой разработки и отладки программного кода. Интуитивно понятное меню позволяет разработчикам быстро освоиться в программе, а со временем, изучая новые функции, улучшать производительность своих работ. Можно выделить два направления развития CCS: для микроконтроллеров (семейства C2000, MSP430, Stellaris, Hercules) и для процессоров (семейства C5000, C6000, Sitara, DaVinci). Последние версии программы базируются на свободном программном обеспечении Eclipse.

Среда включает в себя редактор исходных текстов, отладчик, линковщик, компилятор, средства визуализации, симулятор любого встроенного процессора от Texas Instruments, а также целый набор вспомогательных инструментов. Встроенный отладчик приспособлен к используемым микросхемам, он способен выставлять и работать с многофункциональными контрольными точками. Менеджер событий

останавливает работу центрального процессора или вызывает иные действия, следуя определенным заранее условиям. Среда поддерживает режим трассировки, помогающей проводить отладку программного кода и находить некоторые ошибки «невидимые» для других методов. Также CCS поддерживает работу с многоядерными и многопроцессорными системами. Пример работы среды представлен на рисунке 3.1.

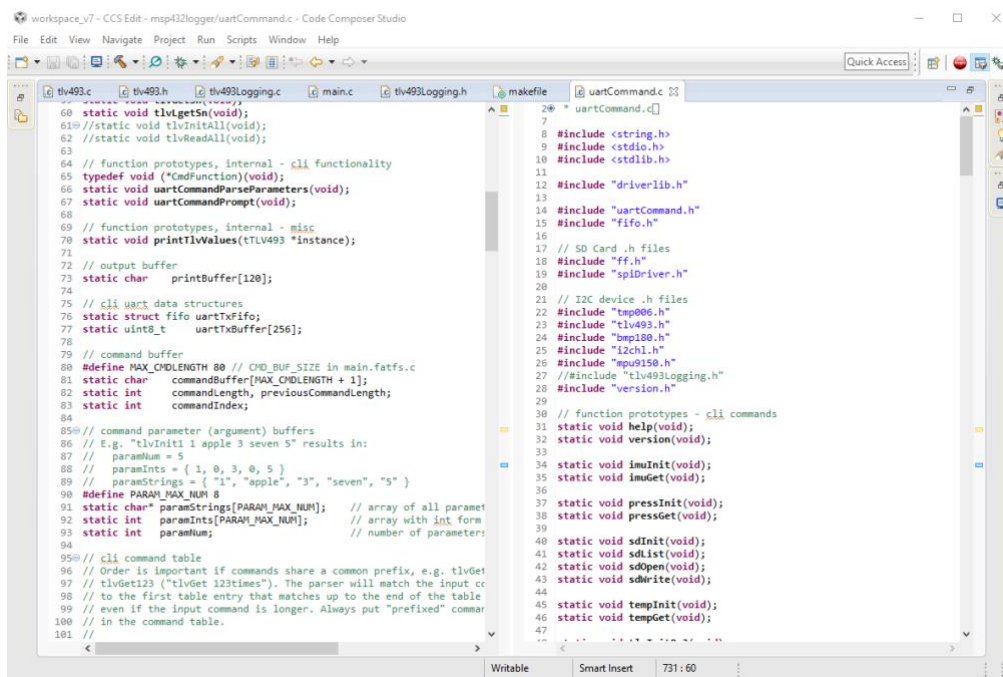


Рисунок 3.1 – Интегрированная среда разработки CSS

3.2 Плата MSP-EXP430F5529

Экспериментальная плата MSP-EXP430F5529 разработана на основе микроконтроллера MSP430F5529 компании Texas Instruments. Это серия процессоров для обработки смешанных сигналов со сверхнизким энергопотреблением. Обобщенная архитектура микроконтроллера представлена на рисунке 3.2.

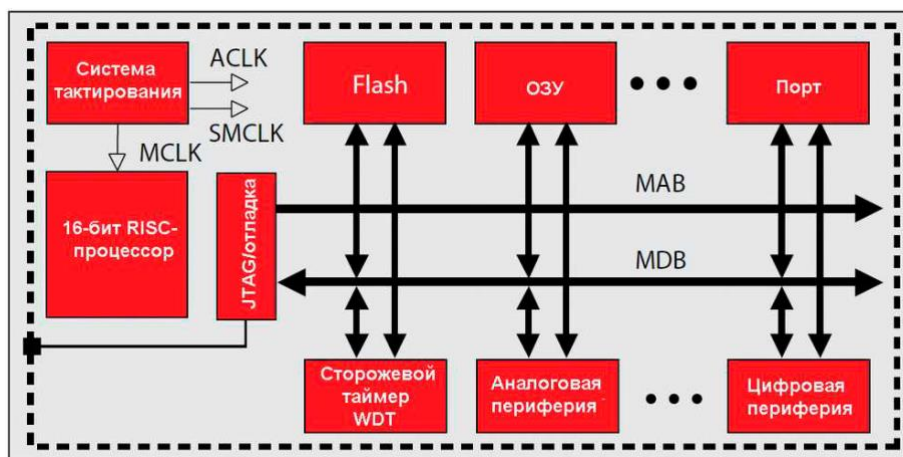


Рисунок 3.2 – Архитектура микроконтроллера MSP430

Плата MSP-EXP430F5529 подключается к USB-порту ПК через разъем ezUSB платы.

Основные особенности архитектуры:

- 16-разрядная ортогональная RISC архитектура;
- Фон-Неймановская адресная шина общей памяти и шина данных памяти;
- 27 (51) команд + 37 расширенных инструкций (20-бит адрес) + 11 адресных инструкций (20-бит операнды, но ограничения в режимах адресации);
- 7 согласованных способов адресации;
- полный программный доступ к регистрам, включая счетчик команд (PC),
- регистр состояния (SR), указатель стека (SP);
- однотактные регистровые операции;
- большой размер регистрового файла, уменьшающий количество обращений к памяти;
- 20-битная шина адреса, 16-битная шина данных;
- генератор констант (6);
- пересылки память-память без промежуточного сохранения в регистре;
- гибкая система тактирования;
- несколько режимов пониженного энергопотребления;
- моментальный переход в активный режим (порядка 6 мкс).

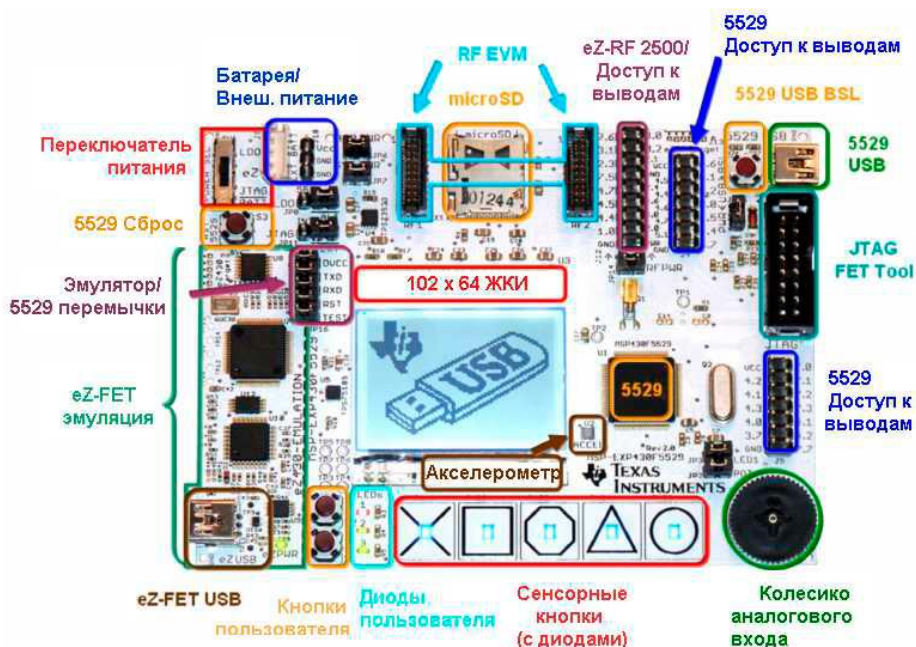


Рисунок 3.3 – Назначение элементов экспериментальной платы MSP-EXP430F5529

Микроконтроллер обладает следующими характеристиками:

- производительность до 25 MIPS;
- напряжение питания 1,8-3,6 В;

- ток утечки вывода 50 нА;
 - потребление в режиме хранения данных 0,1 мкА;
 - потребление в режиме часов реального времени 2,5 мкА.
- Микроконтроллер включает в свой состав:
- флеш-память 128 Кб, SRAM 8 Кб;
 - 80 выводов, 63 линии входа/выхода;
 - асинхронных 16-разрядных таймера/счетчика (7,5,3,3 регистров захвата соответственно);
 - сторожевой таймер (WDT) и таймер часов реального времени (RTC);
 - модуль управления питанием PMM с блоками защиты от падений напряжения (BOR) и контроля напряжения питания (SVS);
 - универсальный последовательный коммуникационный интерфейс USCI 2x UART/LIN/IrDA/SPI + 2 x I2C/SPI;
 - канала DMA;
 - умножитель-накопитель MPY 32 x 32 бита;
 - компаратор;
 - 12 разрядный АЦП (ADC 12A), 16 каналов;
 - полноскоростной USB 2.0 (12Мб/с), до 8 линий в/в со встроенным 3,3 В стабилизатором (питание от 5 В шины, обеспечивает ток 12 мА);
 - интерфейс для измерения линейных и угловых перемещений (SIF);
 - LCD контроллер до 128 сегментов;
 - внутренний генератор частоты с цифровым управлением.

3.3 Цифровой ввод-вывод

8-разрядные порты P1, P2, P3,...,P8, PJ управляют выводами контроллера. Выводы программируются либо как I/O, либо как вход/выход периферии. Порты могут объединяться в пары: P1 и P2 = PA, P3 и P4 = PB, P5 и P6 = PC, P7 и P8 = PD. При работе с прерываниями порты в пары не объединяются. Для порта могут быть доступны регистры:

- RxIN – чтение данных с вывода;
- RxOUT – установка значения выхода;
- RxDIR – выбор направления: 0 – вход, 1 – выход;
- RxREN – разрешение подтягивающего резистора;
- RxDS – выбор допустимой силы вывода;
- RxSEL – выбор функции вывода: 0 – I/O, 1 – периферия;
- RxIV – генерирует значение для изменения счетчика команд, соответствующее прерыванию с максимальным приоритетом;
- RxIES – выбор направления перепада для генерации запроса на прерывание: 0 – по фронту, 1 – по спаду;
- RxIE – разрешение прерывания;
- RxIFG – флаг прерывания.

Адреса соответствующих портов представлены в таблице 3.1.

Таблица 3.1 — Адреса портов ввода-вывода

№ порта	1	2	3	4	5	6	7	8	J
База	200h		0220h		0240h		0260h		0320h
PxIN	0	1	0	1	0	1	0	1	0
PxOUT	2	3	2	3	2	3	2	3	2
PxDIR	4	5	4	5	4	5	4	5	4
PxREN	6	7	6	7	6	7	6	7	6
PxDS	8	9	8	9	8	9	8	9	8
PxSEL	A	B	A	B	A	B	A	B	-
PxIV	E	1E	-	-	-	-	-	-	-
PxIES	18	19	-	-	-	-	-	-	-
PxIE	1A	1B	-	-	-	-	-	-	-
PxIFG	1C	1D	-	-	-	-	-	-	-

3.4 Дребезг контактов

Дребезг контактов — это явление, которое возникает при замыкании или размыкании контактных групп в механических переключателях, кнопках или реле. При этом, когда контакт замыкается, может возникать серия быстрых, коротких замыканий и размыканий, что приводит к ошибочным или многократным сигналам. Этот эффект возникает из-за механического и электрического инерционного поведения контактов.

Дребезг контактов возникает по нескольким причинам: механическая инерция, когда дребезг возникает из-за их формы, материала или недостаточной жесткости пружины; электрический шум, когда кратковременные замыкания и размыкания создают помехи в электронных схемах; и высокая скорость переключения, при которой инерция не позволяет контактам быстро стабилизироваться в одном положении. Он вызывает проблемы, например, множественные сигналы при одном физическом нажатии, когда система может принять несколько срабатываний кнопки за отдельные события, что приводит к ошибкам в работе, поскольку схема не отличает реальный сигнал от дребезга.

4. ВЫПОЛНЕНИЕ РАБОТЫ

4.1 Описание исходного текста программы

В строках 7 объявляется параметр `NUM_CHECKS` — количество проверок. Константы `RED_MODE`, `YELLOW_MODE` и `BOTH_MODE` определяют режимы работы светодиодов: только красный, только желтый и оба светодиода. `NUM_MODES` задает количество режимов. Константы `INACTIVE` и `ACTIVE` обозначают состояния кнопки: не нажата и нажата. Константа `TOGGLE_DELAY` обозначает задержку мигания.

В строке 16 определяется задержка между проверками состояния кнопки (`debounce_delay`), а в строке 17 — минимальное количество "валидных" нажатий (`min_valid_clicks`), необходимое для определения того, что кнопка действительно нажата. Переменная `mode` (строка 19) хранит текущий режим светодиодов, который по умолчанию установлен в `BOTH_MODE`, а `btn_prev_state` (строка 20) сохраняет предыдущее состояние кнопки.

В строках 23-28 объявляются прототипы функций: `delay` для создания задержки, `debounce_button` для проверки состояния кнопки с устранениемдребезга, `toggle_current_mode` для переключения состояния светодиодов в зависимости от текущего режима, и `next_mode` для перехода к следующему режиму работы светодиодов. Также `handle_button_press` для обработки нажатия кнопки и переключения режима и `manage_led_mode` для управления состоянием светодиодов в зависимости от выбранного режима.

Основная функция `main` начинается с отключения сторожевого таймера в строке 32, чтобы избежать перезагрузки микроконтроллера. В строках 33 и 34 настраиваются порты для светодиодов: P1.0 для красного светодиода и P8.1 для желтого. В строке 35 порт P1.7 настраивается как вход для кнопки. В строках 36 и 37 активируется подтягивающий резистор на этом порту, чтобы обеспечить высокое состояние, когда кнопка не нажата. Далее, в строках 38 и 39 включаются оба светодиода для начального состояния.

Основной цикл программы начинается в строке 41. Внутри цикла проверяется текущий режим светодиодов. Если режим не равен `BOTH_MODE`, вызывается функция `manage_led_mode` для переключения состояния светодиодов.

Функция `delay` (строки 51-55) создает задержку на заданное количество миллисекунд. Внутри этой функции используется цикл, где переменная `counter` уменьшается на каждом шаге, что создает необходимую задержку.

В строках 57-63 реализована функция `handle_button_press`. Она проверяет состояние кнопки с устранениемдребезга. Если кнопка нажата, вызывается функция `next_mode` для перехода к следующему режиму.

В строках 65-77 реализована функция `manage_led_mode`. Она переключает состояние светодиодов в текущем режиме и обрабатывает нажатие кнопки, чтобы сменить режим, если кнопка была нажата.

Функция `next_mode` (строки 50-53) переключает режим работы светодиодов.

В строках 87-110 реализована функция `debounce_button` для устранениядребезга кнопки. При обнаружении перехода входа P1.7 из высокого уровня в низкий (предполагаемое нажатие кнопки) выполняется серия из `NUM_CHECKS` (10) опросов входа с паузой `debounce_delay` (15 условных тактов) между ними.

Для каждого опроса, при котором уровень остаётся низким, увеличивается счётчик `count_valid`. Если количество стабильных низких уровней достигает порогового значения `min_valid_clicks`, состояние кнопки считается устойчиво нажатым. Событие формируется только при последующем переходе входа P1.7 в высокое состояние (отпускание кнопки), что позволяет избежать ложных срабатываний придребезге.

В строках 90-98 проверяется, что кнопка остается нажата определенное количество раз в течение проверок. Если это условие выполняется, регистрируется нажатие кнопки. В строках 105-108 обрабатывается отпускание кнопки.

В строках 113-130 реализована функция `toggle_current_mode`, которая управляет состоянием светодиодов в зависимости от текущего режима. В режиме только красного светодиода переключается его состояние. В режиме только желтого — переключается желтый светодиод. В режиме обоих светодиодов оба светодиода включаются.

4.2 Исходный текст программы

```
001 // гр. 250541 Власов Роман Евгеньевич
002 // лабораторная работа №1
003 // Тема: «Среда разработки Code Composer Studio. Плата MSP-EXP430F5529.
    Цифровой ввод-вывод»
004
005 #include <msp430.h> // Заголовок с определениями регистров/битов и
    периферии MSP430
006
007 #define NUM_CHECKS 10 // Общее количество проверок для антидребезга
008 #define RED_MODE 0 // Режим только красного светодиода
009 #define YELLOW_MODE 1 // Режим только желтого светодиода
010 #define BOTH_MODE 2 // Режим обоих светодиодов
011 #define NUM_MODES 3 // Количество режимов
012 #define INACTIVE 0 // Кнопка не нажата
013 #define ACTIVE 1 // Кнопка нажата
014 #define TOGGLE_DELAY 10000 // Задержка мигания
015
016 const unsigned int debounce_delay = 15; // Задержка между проверками
    кнопки
017 const unsigned int min_valid_clicks = NUM_CHECKS * 70 / 100; //
    Минимальное количество состояний кнопки при изменении фронта
018 // (глобальные переменные состояния)
019 unsigned int mode = BOTH_MODE; // Начальный режим
```



```

020 unsigned int btn_prev_state = INACTIVE; // Предыдущее состояние кнопки
021
022 // Прототипы функций
023 void delay(unsigned int cycles); // Функция для создания задержки в
    миллисекундах
024 unsigned char debounce_button(void); // Функция проверки нажатия кнопки
    с устранением дребезга
025 void toggle_current_mode(void); // Функция переключения состояния
    светодиодов в зависимости от текущего режима
026 void next_mode(void); // Функция для перехода к следующему режиму
    светодиодов
027 void handle_button_press(void); // Функция обработки нажатия кнопки и
    переключения режима
028 void manage_led_mode(void); // Функция управления состоянием светодиодов
    в зависимости от выбранного режима
029
030 int main(void) // Точка входа программы
031 {
032     WDTCTL = WDTPW | WDTHOLD; // Отключение сторожевого таймера
033     P1DIR |= BIT0; // Устанавливаем P1.0 как выход для красного
    светодиода
034     P8DIR |= BIT1; // Устанавливаем P8.1 как выход для желтого
    светодиода
035     P1DIR &= ~BIT7; // Установка BIT7 порта P1 на ввод
036     P1REN |= BIT7; // Подтягивающий резистор
037     P1OUT |= BIT7; // Установка подтягивающего резистора к Vcc, чтобы
    при отсутствии нажатия кнопки вход был высоким
038     P1OUT |= BIT0; // Включение красного светодиода
039     P8OUT |= BIT1; // Включение желтого светодиода
040
041     for(;;) // Бесконечный цикл
042     {
043         if (mode == BOTH_MODE) // Проверка: выбран ли режим постоянного
            свечения обоих светодиодов
044             handle_button_press(); //Обработка кнопки
045         else // Иначе: один из режимов мигания
046             manage_led_mode(); // Мигаем соответствующим светодиодом и
            следим за кнопкой
047     }
048 }
049
050 // Задержка
051 void delay(unsigned int cycles) // Параметр cycles задаёт длительность
    пустого цикла
052 {
053     volatile unsigned long counter; // Volatile, чтобы оптимизатор не
    удалил пустой цикл
054     for (counter = cycles ; counter > 0; --counter); // Пустой цикл для
    программной паузы
055 }
056
057 // Обработка нажатия кнопки
058 void handle_button_press(void) {
059     if (debounce_button()) // Если кнопка была нажата
060     { // Начало блока реакции на событие
061         next_mode(); // Переход на следующий режим
062     }
063 }
064
065 // Управление режимом светодиодов
066 void manage_led_mode(void) {
067     toggle_current_mode(); // Один шаг мигания (инвертировать целевой
    светодиод)
068     unsigned int i = 0; // Счётчик ожидания/опроса

```

```

069     for (i = 0; i < TOGGLE_DELAY; ++i)
070     {
071         if (debounce_button()) // Проверка отпущания
072         {
073             next_mode(); // Переключение на следующий режим
074             if (mode == BOTH_MODE) toggle_current_mode(); // Включить
оба светодиода
075             break; // Выход из ожидания после смены режима
076         }
077     }
078 }
079
080 // Переключение режимов светодиодов
081 void next_mode(void) {
082     mode = ++mode % NUM_MODES; // Циклический переход режимов: 0-1-2-0
083     P1OUT &= ~BIT0; // Выключение красного светодиода
084     P8OUT &= ~BIT1; // Выключение желтого светодиода
085 }
086
087 // Проверка нажатия кнопки с устранениемдребезга
088 unsigned char debounce_button(void)
089 {
090     if (!(P1IN & BIT7) && !btn_prev_state) // Обнаружено нажатие (низкий
уровень) при предыдущем состоянии «отпущена»
091     {
092         unsigned int count_valid = 0;
093         unsigned int i;
094         for (i = 0; i < NUM_CHECKS; ++i) { // Многократный опрос для
подавлениядребезга
095             delay(debounce_delay); // Пауза между выборками
096             if (!(P1IN & BIT7)) // Если по-прежнему низкий уровень
(кнопка удерживается)
097                 ++count_valid; // Учитываем стабильный «0»
098         }
099         if (min_valid_clicks <= count_valid) // Достигнут порог
стабильности по «нажатию»
100         {
101             btn_prev_state = ACTIVE; // Запоминаем состояние «кнопка
нажата»
102             return INACTIVE; // Событие клика пока не формируем (ждём
отпущания)
103         }
104     }
105     else if ((P1IN & BIT7) && btn_prev_state) // Обнаружено отпущание
(высокий уровень) при предыдущем состоянии «нажата»
106     {
107         btn_prev_state = INACTIVE; // Возврат к состоянию «кнопка
отпущена»
108         return ACTIVE; // Формируем событие (клик завершён)
109     }
110     return INACTIVE; // По умолчанию — события нет
111 }
112
113 // Переключение светодиодов в зависимости от режима
114 void toggle_current_mode(void)
115 {
116     switch (mode) // Выбор действия по текущему режиму
117     {
118         case RED_MODE:
119             P1OUT ^= BIT0; // Переключение красного светодиода
120             break;
121         case YELLOW_MODE:
122             P8OUT ^= BIT1; // Переключение желтого светодиода
123

```

```

124             break;
125
126         case BOTH_MODE:
127             P1OUT |= BIT0; // Включение красного светодиода
128             P8OUT |= BIT1; // Включение желтого светодиода
129             break;
130     }
131 }

```

5. ВЫВОДЫ

В ходе выполнения лабораторной работы была освоена интегрированная среда разработки Code Composer Studio. Рассмотрен процесс создания проекта, его компиляции и загрузки программы в микроконтроллер, а также базовые приёмы отладки. Подробно изучены основные функциональные возможности экспериментальной платы MSP-EXR430F5529, в том числе организация портов ввода-вывода, настройка линий на режим входа/выхода, применение подтягивающего резистора для работы с кнопкой.

Была разработана программа управления двумя светодиодами и кнопкой согласно заданному алгоритму, обеспечивающая несколько режимов работы индикации и их переключение по отпусканию кнопки. Отдельно была реализована и протестирована программная обработка дребезга контактов, что позволило обеспечить устойчивую работу схемы без ложных срабатываний.